LeetCode Practice

16 Easy: Two Pointers

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

16
EASY

Patterns (in order)

1. Two Pointers (16)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

16 questions | tap a title to jump to it

Easy questions (16) Go >

Two Pointers (16) Go >

E	1. Remove Duplicates from Sorted Array	LC 26
E	2. Remove Element	LC 27
E	3. Find the Index of the First Occurrence in a String	LC 28
E	4. Merge Sorted Array	LC 88
E	5. Valid Palindrome	LC 125
E	6. Happy Number	LC 202
E	7. Move Zeroes	LC 283
E	8. Reverse String	LC 344
E	9. Intersection of Two Arrays II	LC 350
E	10. Is Subsequence	LC 392
E	11. Valid Word Abbreviation	LC 408
E	12. Assign Cookies	LC 455
E	13. Valid Palindrome II	LC 680
E	14. Count Binary Substrings	LC 696
E	15. Squares of a Sorted Array	LC 977
E	16. Merge Strings Alternately	LC 1768

Easy questions (16)

EASY | TOPIC 1 OF 1

Two Pointers

16 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Remove Duplicates from Sorted Array LC 26
2. Remove Element LC 27
3. Find the Index of the First Occurrence in a String LC 28
4. Merge Sorted Array LC 88
5. Valid Palindrome LC 125
6. Happy Number LC 202
7. Move Zeroes LC 283
8. Reverse String LC 344
9. Intersection of Two Arrays II LC 350
10. Is Subsequence LC 392
11. Valid Word Abbreviation LC 408
12. Assign Cookies LC 455
13. Valid Palindrome II LC 680
14. Count Binary Substrings LC 696
15. Squares of a Sorted Array LC 977
16. Merge Strings Alternately LC 1768

EASY | Two Pointers | LeetCode #26 | 1 of 16 | 1/16

Remove Duplicates from Sorted Array

QUESTION

Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of unique elements in nums to be k. After removing duplicates, return the number of unique elements k.

The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer wi
th correct length

int k = removeDuplicates(nums); // Calls your impleme
ntation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1

Input	`nums = [1,1,2]`
Output	`2, nums = [1,2,_]`
Explanation	*Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively. It does not matter what you leave beyond the returned k (hence they are underscores).*

Example 2

Input	`nums = [0,0,1,1,1,2,2,3,3,4]`
Output	`5, nums = [0,1,2,3,4,_,_,_,_,_]`
Explanation	*Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively. It does not matter what you leave beyond the returned k (hence they are underscores).*

CONSTRAINTS

- $1 <= nums.length <= 3 * 10^4$
- $100 <= nums[i] <= 100$
- nums is sorted in non-decreasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have a sorted integer array and we need to remove duplicates in-place

so each value appears only once. We return k — the count of unique elements —

and the first k slots of nums must hold those unique values in sorted order. Right?"

"A few quick questions:

Can values be negative? Any constraint on the range?

The elements beyond index k don't matter — we just need the first k to be correct?

Is the array guaranteed to be non-empty?"

"Let me walk the example: nums=[1,1,2].

After processing: nums=[1,2,_], k=2. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
unique = []
for x in xes:
    if x is not already in unique:
        append it
    copy unique back into nums
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

The membership test scans the list, O(n) each. The array is sorted, so duplicates are adjacent: a write pointer that only keeps a value different from the last kept one is O(n) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the sort and set — both unnecessary because nums is already sorted.

In a sorted array duplicates are always adjacent, so a single write pointer k is enough.

Start k at 1. Walk forward: whenever nums[i] differs from nums[k-1], it's a new unique —

write it at nums[k] and advance k. Elements equal to nums[k-1] are simply skipped.

Pseudocode:

```
k = 1
for i from 1 to len(nums)-1:
if nums[i] != nums[k-1]:
nums[k] = nums[i]; k++
return k
```

PHASE 3 OPTIMIZE

Time: O(n) — one pass. Space: O(1) — two integer pointers. Does that make sense?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _26_RemoveDuplicatesSortedArray:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 1
        for i in range(1, len(nums)):
            if nums[i] != nums[k - 1]:
                nums[k] = nums[i]
                k += 1
        return k
```

PHASE 5 TEST & DEBUG

"Let me trace removeDuplicates([1,1,2])."

slow=0 (insertion pointer). Fast scans from 1.

fast=1: nums[1]=1==nums[0]=1 -> skip. fast=2: nums[2]=2!=nums[1]=1 -> slow++=1, nums[1]=2.

Array becomes [1,2,2]. Return slow+1=2 ok (first 2 elements are unique).

[0,0,1,1,1,2,2,3,3,4]: slow=0,fast=1: dup. fast=2: 1!=0->nums[1]=1,slow=1. fast=3: dup. fast=4: dup.

fast=5: 2!=1->nums[2]=2,slow=2. fast=7: 3!=2->nums[3]=3,slow=3. fast=9: 4!=3->nums[4]=4,slow=4.

Return 5 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one forward pass, each element visited once	the write pointer k and loop index i are the only extras

The key insight: sorted order guarantees duplicates are adjacent, so a single write pointer is all we need — no hashing, no sorting.

Follow-up: Remove Duplicates II (LC 80) — allow at most 2 of each.

Same pattern but compare against nums[k-2] instead of nums[k-1].

Follow-up: generalise to 'at most m copies' — compare nums[i] vs nums[k-m].

EASY | Two Pointers | LeetCode #27 | 2 of 16 | 2/16

Remove Element

QUESTION

Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return the number of elements in nums which are not equal to val.

Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:

CONSTRAINTS

- Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.
- Return k.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                            // It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1

Input	`nums = [3,2,2,3], val = 3`
Output	`2, nums = [2,2,_,_]`
Explanation	*Your function should return k = 2, with the first two elements of nums being 2. It does not matter what you leave beyond the returned k (hence they are underscores).*

Example 2

Input	`nums = [0,1,2,2,3,0,4,2], val = 2`
Output	`5, nums = [0,1,4,0,3,_,_,_]`
Explanation	*Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4. Note that the five elements can be returned in any order. It does not matter what you leave beyond the returned k (hence they are underscores).*

QUESTION

- 0 <= nums.length <= 100
- 0 <= nums[i] <= 50
- 0 <= val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We're given an array and a value val, and we need to remove every occurrence

of val in-place. We return k — the count of elements that are NOT val —

and the first k slots of nums must hold those non-val elements. Order can change. Right?"

"A few quick questions:

Can the array be empty — would we return k=0?

If val doesn't appear at all, we return len(nums)?

Does it matter which order the kept elements appear in?"

"Let me trace the example: nums=[3,2,2,3], val=3.

We want k=2 and the first two slots to be the two 2s. ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    if nums[i] == val:
        shift every later element one place
            left and shrink the array
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each removal shifts up to n elements. A write pointer that copies only the elements not equal to val is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the extra list. I can avoid it entirely with a write pointer k.

Walk through nums: whenever nums[i] != val, write it at nums[k] and advance k.

Elements equal to val are simply skipped — the write pointer never moves for them.

Pseudocode:

```
k = 0
for each num in nums:
if num != val: nums[k] = num; k++
```

PHASE 3 OPTIMIZE

```
return k
```

Time: O(n) — one pass. Space: O(1) — one pointer. Does that make sense?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach.

```
class _27_RemoveElement:
    def removeElement(self, nums: List[int], val: int) ->
int:
        k = 0
        for num in nums:
            if num != val:
                nums[k] = num
                k += 1
        return k
```

PHASE 5 TEST & DEBUG

"Let me trace removeElement([3,2,2,3], val=3)."

slow=0. fast scans:

fast=0: nums[0]=3==val -> skip. fast=1: nums[1]=2!=3 -> nums[0]=2, slow=1.

fast=2: nums[2]=2!=3 -> nums[1]=2, slow=2. fast=3: nums[3]=3==val -> skip.

Result: [2,2,2,3] (first 2 elements: [2,2]). Return k=2 ok

[0,1,2,2,3,0,4,2], val=2: k=5, array=[0,1,3,0,4,_,_,_] ok

WHY slow pointer: only advance slow when we keep an element — it's the write cursor.

WHY in-place with slow pointer: O(1) space. Slow=write cursor. Fast=read cursor. k-pass O(n) total.

Result array first k elements are the kept values (judge validates only first k positions).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single forward pass	one write pointer

Since order doesn't need to be preserved, there's also a two-pointer variant:

when we find val at left, swap it with nums[right] and shrink right.

That approach minimises write operations when val is rare — useful if writing is expensive.

Follow-up: Remove Duplicates (LC 26) — same write-pointer, different skip condition.

Follow-up: Move Zeroes (LC 283) — same idea, but zeros go to the back after the kept elements.

EASY | Two Pointers | LeetCode #28 | 3 of 16 | 3/16

Find the Index of the First Occurrence in a String

QUESTION

Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1

Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Explanation	*"sad" occurs at index 0 and 6. The first occurrence is at index 0, so we return 0.*

Example 2

Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Explanation	*"leeto" did not occur in "leetcode", so we return -1.*

CONSTRAINTS

- 1 <= haystack.length, needle.length <= 10^4
- haystack and needle consist of only lowercase English characters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the first occurrence of needle in haystack. Return -1 if not found.

Must return the starting index. Right?"

"A few quick questions:

Can needle be empty? Per constraints length >= 1.

Can haystack be shorter than needle? Yes — return -1."

"Let me walk: haystack='sadbutsad', needle='sad' -> 0 ok. needle='but' -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    compare needle with haystack[i..i + m]
return the first i that matches
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

After a mismatch it throws away what it learned and re-reads characters. KMP (or Z-function) is O(n + m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n*m) substring comparison at each position.

Python's built-in str.find() uses optimised C-level algorithms internally.

For a pure implementation, KMP runs in O(n+m) using a failure function table.

For the interview, Python's find() is acceptable and mention KMP as a follow-up.

Time: O(n+m) with KMP. Space: O(m) for the failure table."

Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

Key insight: KMP's failure function precomputes a "mismatch jump table" — each character is visited at most twice total, giving O(n+m) vs O(nm) naive.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _28_FindIndexFirstOccurrence:
    def strStr(self, haystack, needle):
        return haystack.find(needle)
```

PHASE 5 TEST & DEBUG

"Let me trace strStr on haystack='hello', needle='ll'."

Sliding window: n=5, m=2

i=0: haystack[0:2]='he' != 'll'

i=1: haystack[1:3]='el' != 'll'

i=2: haystack[2:4]='ll' == 'll' -> return 2 ok

Edge: needle='' -> return 0 per problem spec

WHY: simple O(n*m) slice; KMP improves to O(n+m) for repeated patterns.

Call trace: compare substring at each position until match found.

Return 2 means 'll' starts at index 2 in 'hello'. ok

PHASE 6 COMPLEXITY & FOLLOW-UP

Brute O(n*m). Python find() O(n+m) internally. KMP O(n+m) with O(m) space.

For an interview, state 'Python uses Boyer-Moore-Horspool internally' and offer KMP.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: KMP algorithm — build failure function, then slide with O(1) per char.

Follow-up: Repeated Substring Pattern (#459) — uses KMP failure function on 2s.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #88 | 4 of 16 | 4/16

Merge Sorted Array

QUESTION

You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n, representing the number of elements in nums1 and nums2 respectively.

Merge nums1 and nums2 into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array nums1. To accommodate this, nums1 has a length of m + n, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1

Input	`nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3`
Output	`[1,2,2,3,5,6]`
Explanation	*The arrays we are merging are [1,2,3] and [2,5,6].* *The result of the merge is [* *1* *,* *2* *,2,* *3* *,5,6] with the underlined elements coming from nums1.*

Example 2

Input	`nums1 = [1], m = 1, nums2 = [], n = 0`
Output	`[1]`
Explanation	*The arrays we are merging are [1] and [].* *The result of the merge is [1].*

Example 3

Input	`nums1 = [0], m = 0, nums2 = [1], n = 1`
Output	`[1]`
Explanation	*The arrays we are merging are [] and [1].* *The result of the merge is [1].* *Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.*

CONSTRAINTS

- nums1.length == m + n
- nums2.length == n
- 0 <= m, n <= 200
- 1 <= m + n <= 200
- 10^9 <= nums1[i], nums2[j] <= 10^9

QUESTION

- Follow up: Can you come up with an algorithm that runs in O(m + n) time?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

nums1 has m valid elements followed by n zeros as padding.

nums2 has n elements. Both are sorted in non-decreasing order.

We need to merge them in-place into nums1 — sorted, no return value. Right?"

"A few quick questions:

Can m or n be 0? Yes — if n=0 nums1 is already done; if m=0 we just copy nums2.

Can there be negative numbers? Yes, constraints go down to -10^9.

Can there be duplicates? Yes — non-decreasing order means equal values are allowed.

We cannot use extra space for the merge? The follow-up asks for O(m+n) time,

implying in-place. I'll target O(1) extra space."

"Let me walk example 1:

nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3.

Merge [1,2,3] and [2,5,6] into nums1 -> [1,2,2,3,5,6]. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
copy nums2 into the empty tail of nums1
sort nums1
```

TIME	SPACE
O((m + n) log(m + n))	O(1)

WHY IT IS SLOW

Ignores that both halves are already sorted. Merging from the back with three pointers (so nothing is overwritten before it is read) is O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force throws away the sorted order and pays O(log(m+n)) to re-sort.

The classic merge from the front copies to a temp array — O(m+n) space.

But there's a smarter way that stays O(1) space.

Key insight: if we merge from the BACK, we never overwrite elements we still need.

PHASE 3 OPTIMIZE

The tail of nums1 is empty padding — we can fill it safely from right to left.

Three pointers:

i -> last valid element in nums1 (index m-1)

j -> last element in nums2 (index n-1)

k -> last slot in nums1 (index m+n-1)

At each step, pick the larger of nums1[i] and nums2[j], place it at k, move that pointer left.

When j drops below 0, nums2 is exhausted — nums1's remaining elements are already in place.

When i drops below 0 first, copy remaining nums2 into nums1.

Pseudocode:

i, j, k = m-1, n-1, m+n-1

```
while k >= 0:
if j < 0 or (i >= 0 and nums1[i] > nums2[j]):
nums1[k] = nums1[i]; i -= 1
else:
nums1[k] = nums2[j]; j -= 1
k -= 1
```

Time: O(m+n) — each element is placed exactly once.

Space: O(1) — purely in-place.

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _88_MergeSortedArray:
    def merge(self, nums1: List[int], m: int, nums2: List[
int], n: int) -> None:
        last1    = m - 1        # tail of valid nums1 elem
ents
        last2    = n - 1        # tail of nums2
        fill     = m + n - 1   # next slot to fill in nums
1 (right to left)

        while fill >= 0:
            if last2 < 0 or (last1 >= 0 and nums1[last1] >
 nums2[last2]):
                nums1[fill] = nums1[last1]
                last1 -= 1
            else:
                nums1[fill] = nums2[last2]
                last2 -= 1
            fill -= 1
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

"Let me trace through all three examples."

nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3

last1=2, last2=2, fill=5

fill=5: nums1[2]=3 vs nums2[2]=6 -> 6 wins -> nums1[5]=6, last2=1, fill=4

fill=4: nums1[2]=3 vs nums2[1]=5 -> 5 wins -> nums1[4]=5, last2=0, fill=3

fill=3: nums1[2]=3 vs nums2[0]=2 -> 3 wins -> nums1[3]=3, last1=1, fill=2

fill=2: nums1[1]=2 vs nums2[0]=2 -> equal, take nums2 -> nums1[2]=2, last2=-1, fill=1

fill=1: last2<0 -> take nums1[1]=2 -> nums1[1]=2, last1=0, fill=0

fill=0: last2<0 -> take nums1[0]=1 -> nums1[0]=1, last1=-1, fill=-1

nums1=[1,2,2,3,5,6] ok

nums1=[1] m=1, nums2=[] n=0

last1=0, last2=-1, fill=0

fill=0: last2<0 -> nums1[0]=nums1[0]=1. fill=-1. done.

nums1=[1] ok

nums1=[0] m=0, nums2=[1] n=1

last1=-1, last2=0, fill=0

fill=0: last1<0 -> nums1[0]=nums2[0]=1, last2=-1, fill=-1. done.

nums1=[1] ok

All three cases pass including the m=0 and n=0 edges

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m+n)	**O(1)**
each of the m+n positions is filled exactly once	purely in-place, no auxiliary arrays

The core insight worth stating explicitly: merging front-to-back requires a temp

buffer because we'd overwrite nums1's valid elements. Merging back-to-front avoids

this because the padding zeros are exactly the space we need, and large elements

fill the tail first without touching anything we still care about.

Edge cases worth calling out:

— n=0: loop runs but last2<0 always, so nums1's valid elements stay in place.

— m=0: last1<0 always, so nums2 is copied straight into nums1.

— All nums2 elements are larger than all nums1 elements: nums2 fills the back,

then nums1 elements slot into the front — handled cleanly by the pointer logic.

Given more time I'd ask: what if nums1 has exactly m slots (no padding)?

Then in-place is impossible — we'd need to return a new merged array,

which is the standard merge step in merge sort.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #125 | 5 of 16 | 5/16

Valid Palindrome

QUESTION

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string s, return true if it is a palindrome, or false otherwise.

Example 1	
Input	`s = "A man, a plan, a canal: Panama"`
Output	`true`
Explanation	*"amanaplanacanalpanama" is a palindrome.*

Example 2	
Input	`s = "race a car"`
Output	`false`
Explanation	*"raceacar" is not a palindrome.*

Example 3	
Input	`s = " "`
Output	`true`
Explanation	*s is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.*

CONSTRAINTS

- $1 <= s.length <= 2 * 10^5$
- s consists only of printable ASCII characters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We strip everything except letters and digits, lowercase it all,

then check if the result reads the same forwards and backwards. Right?"

"A few quick questions:

Alphanumeric means letters AND digits — not just letters?

So '0P' after cleaning is '0p', which is not a palindrome — confirmed?

What about a string of only spaces or punctuation?

Example 3 shows a single space returns true — after cleaning it's an empty string,

and an empty string is considered a palindrome. Good.

PHASE 1 CLARIFY

Can the string be length 1? Yes — a single alphanumeric character is a palindrome."

"Let me walk example 1:

'A man, a plan, a canal: Panama' -> 'amanaplanacanalpanama' -> palindrome ok

Example 2: 'race a car' -> 'raceacar' -> not a palindrome ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
t = the lowercase letters and digits of s only
return t == reverse(t)
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but two extra copies of the string. Two pointers moving inward, skipping punctuation, need O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force is already O(n) time, but uses O(n) space for the cleaned copy.

We can do this in O(1) extra space using two pointers directly on the original string.

Place left at 0 and right at len(s)-1.

Advance left past any non-alphanumeric characters.

Retreat right past any non-alphanumeric characters.

Compare s[left].lower() and s[right].lower().

If they differ, return False. Otherwise move both pointers inward.

If left meets or passes right, the string is a palindrome.

The key benefit: we never build a second string — we skip junk characters on the fly.

Pseudocode:

left, right = 0, len(s)-1

```
while left < right:
while left < right and not s[left].isalnum(): left += 1
while left < right and not s[right].isalnum(): right -=
  1
if s[left].lower() != s[right].lower(): return False
left += 1; right -= 1
return True
```

Time: O(n). Space: O(1). Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _125_ValidPalindrome:
    def isPalindrome(self, s: str) -> bool:
        left  = 0
        right = len(s) - 1

        while left < right:
            while left < right and not s[left].isalnum():
  # skip non-alphanumeric
                left += 1
            while left < right and not s[right].isalnum():
  # skip non-alphanumeric
                right -= 1

            if s[left].lower() != s[right].lower():
                return False

            left  += 1
            right -= 1

        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isPalindrome('A man, a plan, a canal: Panama')."

Filter alphanumeric, lowercase: 'amanaplanacanalpanama'.

Two pointers: left=0, right=20.

left='a', right='a' ok. left='m', right='m' ok. ... All match.

Return True ok

'race a car': filtered='raceacar'. left='r',right='r'ok. left='a',right='a'ok. left='c',right='c'ok. left='e'!='a'->False ok

WHY filter+lowercase first: then standard palindrome two-pointer on clean string.

'': empty string after filtering -> True (vacuously palindrome) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
each character is visited at most once by left or right pointer	no auxiliary storage beyond two integer pointers

The brute force one-liner is perfectly valid for an interview too — it's readable

and still O(n) time. The two-pointer version wins purely on space: O(1) vs O(n).

If the interviewer asks about the follow-up, lead with the space trade-off.

`isalnum() handles both letters and digits in one call — worth naming explicitly`

so the interviewer knows you're not just checking isalpha().

Subtle trap: the inner while loops must guard left < right to avoid crossing

pointers — if you forget, you can read out-of-bounds when the string is all

PHASE 6 COMPLEXITY & FOLLOW-UP

non-alphanumeric (like the single-space example).

Given more time I'd ask: what if the string could contain unicode characters

like accented letters (é, ñ)? isalnum() in Python handles unicode correctly

by default, so our solution already works — worth calling out.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #202 | 6 of 16 | 6/16

Happy Number

QUESTION

Write an algorithm to determine if a number n is happy.

A happy number is a number defined by the following process:

CONSTRAINTS

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
- Those numbers for which this process ends in 1 are happy.

Return true if n is a happy number, and false if not.

Example 1	
Input	`n = 19`
Output	`true`
Explanation	$1^2 + 9^2 = 82$ $8^2 + 2^2 = 68$ $6^2 + 8^2 = 100$ $1^2 + 0^2 + 0^2 = 1$

Example 2	
Input	`n = 2`
Output	`false`

- $1 <= n <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly replace n with the sum of squares of its digits. If it reaches 1, it's happy.

If it loops forever without reaching 1, it's not happy. Right?"

"A few quick questions:

n is positive? Yes. Need to detect cycles to avoid infinite loop."

"Let me walk: n=19 -> $1^2+9^2=82$ -> $8^2+2^2=68$ -> $6^2+8^2=100$ -> $1^2+0+0=1$ -> happy ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
seen = a set
while n != 1 and n not in seen:
    add n to seen
    n = sum of the squares of its digits
return n == 1
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(log n)	O(log n)

NOTE

Not slow: the sequence falls quickly and then repeats. The set costs memory; Floyd's cycle detection with two running values uses O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Fast/slow pointer (Floyd's cycle detection on digit sums) — O(log n). Walk-through:"

HOW digit sum works: sum_sq_digits(n) = sum of squares of each digit of n.

Floyd's cycle: slow = sum_sq_digits(n). fast = sum_sq_digits(sum_sq_digits(n)).

While slow != fast: slow = f(slow); fast = f(f(fast)).

If slow==1: it's a happy number (cycle ends at 1).

Else: it's in a cycle not containing 1 -> not happy.

WHY Floyd's: non-happy numbers enter a cycle among values < 243 (since $9^2*3=243$ for 3-digit).

The slow/fast pointers detect the cycle without storing a set.

Alternative: hash set — if we see a repeated value, it's not happy.

Time: O(log n). Space: O(1) with Floyd's.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _202_HappyNumber:
    def isHappy(self, n):
        def digit_square_sum(x):
            total = 0
            while x:
                d = x % 10
                total += d * d
                x //= 10
            return total
        slow, fast = n, digit_square_sum(n)
        while fast != 1 and slow != fast:
            slow = digit_square_sum(slow)
            fast = digit_square_sum(digit_square_sum(fast)
)
        return fast == 1
```

PHASE 5 TEST & DEBUG

"Let me trace isHappy(19)."

19 -> $1^2+9^2=1+81=82$

82 -> $8^2+2^2=64+4=68$

68 -> $6^2+8^2=36+64=100$

100 -> $1^2+0^2+0^2=1$ -> return True ok

WHY detect cycle: unhappy numbers loop (e.g., 4->16->37->58->89->145->42->20->4)

PHASE 5 TEST & DEBUG

Use fast/slow pointers or a seen set to detect the cycle.

Edge: n=1 -> immediately True; n=7 -> path 7->49->97->130->10->1 -> True.

"Digit sum helper: repeatedly sum squares of digits until 1 or cycle."

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) per step (digit count), bounded total iterations	**O(1)**

Follow-up: Linked List Cycle (#141) — same Floyd's algorithm on an explicit list.

Follow-up: find the actual cycle length once detected — count steps after meeting point.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #283 | 7 of 16 | 7/16

Move Zeroes

QUESTION

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1	
Input	`nums = [0,1,0,3,12]`
Output	`[1,3,12,0,0]`

Example 2	
Input	`nums = [0]`
Output	`[0]`

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 2^{31} <= nums[i] <= 2^{31} - 1
- Follow up: Could you minimize the total number of operations done?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We need to move all 0s to the end of the array while keeping the relative

order of all non-zero elements. This must be done in-place — no copy. Right?"

"A few quick questions:

Do we need to return anything? No — modify in-place, return nothing.

Can the array be all zeros? Yes — output should still be all zeros.

Can it be all non-zeros? Yes — nothing moves.

Can there be negative numbers? Yes, constraints go down to -2^31."

"Let me walk example 1:

nums=[0,1,0,3,12] -> [1,3,12,0,0].

Non-zeros 1,3,12 keep their relative order and shift to the front.

Zeros fill the tail. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    # from the left
    if nums[i] == 0:
        remove it, shift everything after it
            left, and append a 0 at the end
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each zero shifts up to n elements. A write pointer copies the non-zeros forward, then fills the rest with zeros: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force uses O(n) extra space for the non-zero copy.

We can do this in O(1) space with two pointers.

Use a slow pointer 'place' that tracks where the next non-zero should go.

Scan with a fast pointer 'scan'. Whenever nums[scan] is non-zero, swap it with nums[place] and advance place.

This compacts all non-zeros to the front in their original order, and the swaps naturally push zeros toward the tail.

Pseudocode:

```
place = 0
for scan in range(len(nums)):
if nums[scan] != 0:
nums[scan], nums[place] = nums[place], nums[scan]
place += 1
```

Time: O(n). Space: O(1). Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _283_MoveZeroes:
    def moveZeroes(self, nums: List[int]) -> None:
        place = 0
        for scan in range(len(nums)):
            if nums[scan] != 0:
                nums[scan], nums[place] = nums[place], num
s[scan]
                place += 1
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

"Let me trace through both examples."

nums=[0,1,0,3,12]

PHASE 5 TEST & DEBUG

scan=0: nums[0]=0 -> skip. place=0.

scan=1: nums[1]=1 != 0 -> swap(1,0): [1,0,0,3,12]. place=1.

scan=2: nums[2]=0 -> skip. place=1.

scan=3: nums[3]=3 != 0 -> swap(3,1): [1,3,0,0,12]. place=2.

scan=4: nums[4]=12 != 0 -> swap(4,2): [1,3,12,0,0]. place=3.

Result: [1,3,12,0,0] ok

nums=[0]

scan=0: nums[0]=0 -> skip.

Result: [0] ok

nums=[1,2,3] (no zeros)

Every element is non-zero -> each swaps with itself. place advances each step.

Result: [1,2,3] ok (self-swaps are fine — no change)

Self-swaps when place==scan are harmless

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	two integer pointers

Follow-up asks to minimize operations. The swap approach does a swap

for every non-zero element, even if it's already in the right place.

A minor optimization: only swap when place != scan to skip self-swaps.

But in an interview the current version is clean enough.

Alternative: instead of swapping, just overwrite:

write non-zeros to the front with assignment (not swap), then zero-fill the tail.

Same complexity but fewer writes — useful if writes are expensive.

```
place = 0
for x in nums:
if x != 0: nums[place] = x; place += 1
```

nums[place:] = [0] * (len(nums) - place)

Given more time I'd ask: what if we also need to preserve the positions

of the zeros (not just push them to the end)? That changes the problem

to a stable partition — same two-pointer approach still works.

EASY | Two Pointers | LeetCode #344 | 8 of 16 | 8/16

Reverse String

QUESTION

Write a function that reverses a string. The input string is given as an array of characters s.

You must do this by modifying the input array in-place with O(1) extra memory.

Example 1	
Input	`s = ["h","e","l","l","o"]`
Output	`["o","l","l","e","h"]`

Example 2	
Input	`s = ["H","a","n","n","a","h"]`
Output	`["h","a","n","n","a","H"]`

CONSTRAINTS

- 1 <= s.length <= 10^5
- s[i] is a printable ascii character.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse the character array in-place. No new array — swap pointers. Right?"

"A few quick questions:

Modify s in place, no return value? Yes. Length >= 1? Yes."

"Let me walk: s=['h','e','l','l','o'] -> ['o','l','l','e','h'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
copy = a new array
for i in range(n):
    copy[i] = s[n - 1 - i]
    copy back
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear time but a second array. Two pointers swapping from both ends toward the middle reverse it in place with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer swap from both ends — O(n). Walk-through:"

lo=0, hi=len(s)-1. While lo < hi:

s[lo], s[hi] = s[hi], s[lo]. <- swap

lo++. hi--.

WHY two pointers meet in middle: each swap places two characters in their correct reversed positions.

When lo >= hi: all characters are swapped.

In-place: no extra array needed.

Even-length: lo and hi swap all pairs. Odd-length: middle element stays (lo==hi -> loop ends).

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _344_ReverseString:
    def reverseString(self, s):
        left, right = 0, len(s) - 1
        while left < right:
            s[left], s[right] = s[right], s[left]
            left += 1
            right -= 1
```

PHASE 5 TEST & DEBUG

"Let me trace reverseString(['h','e','l','l','o'])."

Two-pointer: lo=0, hi=4

Swap s[0]s[4]: ['o','e','l','l','h']

lo=1, hi=3: swap s[1]s[3]: ['o','l','l','e','h']

lo=2, hi=2: lo>=hi -> stop

Result: ['o','l','l','e','h'] ok

WHY in-place: O(1) space; two pointers meet in middle after n/2 swaps.

Edge: single char or empty -> no swaps needed, return as-is.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)
n/2 swaps	only two integer pointers

This is the building block for many reversal-based algorithms.

Follow-up: Reverse Words in a String (#151) — reverse words not characters.

Follow-up: Reverse Vowels of a String (#345) — two pointers with conditional swap.

EASY | Two Pointers | LeetCode #350 | 9 of 16 | 9/16

Intersection of Two Arrays II

QUESTION

Given two integer arrays nums1 and nums2, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and you may return the result in any order.

Example 1	
Input	`nums1 = [1,2,2,1], nums2 = [2,2]`
Output	`[2,2]`

Example 2	
Input	`nums1 = [4,9,5], nums2 = [9,4,9,8,4]`
Output	`[4,9]`
Explanation	*[9,4] is also accepted.*

CONSTRAINTS

- 1 <= nums1.length, nums2.length <= 1000
- 0 <= nums1[i], nums2[i] <= 1000
- Follow up:
- What if the given array is already sorted? How would you optimize your algorithm?
- What if nums1's size is small compared to nums2's size? Which algorithm is better?
- What if elements of nums2 are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the intersection of two arrays including duplicates.

If an element appears m times in nums1 and n times in nums2, it appears min(m,n) times. Right?"

"A few quick questions:

Order of result doesn't matter? Yes. Return as array? Yes."

"Let me walk: nums1=[1,2,2,1], nums2=[2,2] -> [2,2] ok. nums1=[4,9,5], nums2=[9,4,9,8,4] -> [4,9] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x in nums1:
    scan nums2 for an unused equal element
    mark it used and add x to the answer
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

Scans the second array for each element. Count one array in a hash map and consume the counts while walking the other: O(n + m) time; sorting both and walking two pointers needs O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n*m) linear scan.

Count frequencies in both arrays using Counter.

For each value, add min(count1, count2) copies to result.

Time: O(n+m). Space: O(min(n,m)) for the Counter of the smaller array."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _350_IntersectionOfTwoArraysII:
    def intersect(self, nums1, nums2):
        from collections import Counter
        count = Counter(nums1)
        result = []
        for num in nums2:
            if count.get(num, 0) > 0:
                result.append(num)
                count[num] -= 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

nums1=[1,2,2,1], nums2=[2,2]: count={1:2,2:2}.

num=2: count[2]=2>0 -> append 2, count[2]=1. num=2: count[2]=1>0 -> append 2, count[2]=0.

-> [2,2] ok

nums1=[1], nums2=[1]: count={1:1}. num=1->append. -> [1] ok

Decrementing the counter ensures we don't use the same element more than available

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(min(n,m))** for Counter of smaller array

Follow-up: if both arrays are sorted, use two pointers O(n+m) time O(1) space.

Follow-up: Intersection of Two Arrays I (#349) — each element appears once; use sets.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #392 | 10 of 16 | 10/16

Is Subsequence

QUESTION

Given two strings s and t, return true if s is a subsequence of t, or false otherwise.

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1	
Input	`s = "abc", t = "ahbgdc"`
Output	`true`

Example 2	
Input	`s = "axc", t = "ahbgdc"`
Output	`false`

CONSTRAINTS

- 0 <= s.length <= 100
- 0 <= t.length <= 10^4
- s and t consist only of lowercase English letters.
- Follow up: Suppose there are lots of incoming s, say s_1, s_2, ..., s_k where k >= 10^9, and you want to check one by one to see if t has its subsequence. In this scenario, how would you change your code?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Is s a subsequence of t? A subsequence maintains relative order but doesn't need to be contiguous.

Right?"

"A few quick questions:

Both can be empty? Yes — empty s is a subsequence of anything.

s can be longer than t? Yes — then return false."

"Let me walk: s='ace', t='abcde' -> match a,c,e in order -> true ok. s='aec' -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for choice in choices:
    # of len(s) positions in t (in order)
    if the letters at those positions spell s:
        return True
return False
```

TIME	SPACE
O(C(n, m) * m)	O(m)

WHY IT IS SLOW

Tries every subset of positions. Walk t once with a pointer into s, advancing it on each match: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer greedy on s and t — O(n). Walk-through:"

i=0 (pointer in s), j=0 (pointer in t).

While i < len(s) and j < len(t):

If s[i] == t[j]: i++. <- matched a character of t, advance both

j++. <- always advance t

Return i == len(s). <- all of s matched in order

WHY greedy: matching the first occurrence of each character of s in t is always optimal.

If we can match s[i] at position j, we should — taking it earlier can only help (more room for rest).

WHY advance j always: t is the string we search through; we scan it left to right.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _392_IsSubsequence:
    def isSubsequence(self, s, t):
        s_ptr = 0
        for ch in t:
            if s_ptr < len(s) and ch == s[s_ptr]:
                s_ptr += 1
        return s_ptr == len(s)
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

s='ace', t='abcde':

a->match s[0], s_ptr=1. b->no. c->match s[1], s_ptr=2. d->no. e->match s[2], s_ptr=3.

s_ptr==len(s)=3 -> True ok

s='axc', t='ahbgdc': a->match. x->h,b,g,d,c all no match. s_ptr=1!=3 -> False ok

PHASE 5 TEST & DEBUG

s='', t='anything': s_ptr=0==0 -> True ok

S_ptr check before comparison handles the case where s is already exhausted

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(|t|)	**O(1)**
one pass through t	one pointer

Follow-up: for k queries each with different s against same t — precompute

```
next[i][c] = next index in t at or after i where char c
  occurs. O(|t|*26) precompute, O(|s|) per query.
```

Follow-up: Longest Common Subsequence (#1143) — generalise to longest, not just check.

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #408 | 11 of 16 | 11/16

Valid Word Abbreviation

QUESTION

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

CONSTRAINTS

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
- "s010n" (has leading zeros)
- "s0ubstitution" (replaces an empty substring)

Given a string word and an abbreviation abbr, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1

Input	`word = "internationalization", abbr = "i12iz4n"`
Output	`true`
Explanation	*The word "internationalization" can be abbreviated as "i12iz4n" ("i nternational iz atio n").*

Example 2

Input	`word = "apple", abbr = "a2e"`
Output	`false`
Explanation	*The word "apple" cannot be abbreviated as "a2e".*

- 1 <= word.length <= 20
- word consists of only lowercase English letters.
- 1 <= abbr.length <= 10
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

An abbreviation replaces one or more non-adjacent non-empty substrings with their lengths.

Given a word and an abbreviation, return true if the abbreviation is valid for the word.

Numbers in the abbreviation represent how many consecutive characters to skip. Right?"

"A few quick questions:

Can there be leading zeros in the number part? No — that's explicitly invalid.

So '01' is not valid even though it could theoretically mean skip 1.

Can the abbreviation be the full word with no numbers? Yes — that's valid.

Can it be just a single number like '12'? Yes — if the word has exactly 12 chars."

"Let me walk example 1:

word='internationalization', abbr='i12iz4n'.

'i' matches 'i'. '12' skips 12 chars. 'i' matches 'i'. 'z' matches 'z'.

'4' skips 4 chars. 'n' matches 'n'. We reach end of both -> true ok

Example 2: word='apple', abbr='a2e'. 'a' matches. '2' skips 'pp'. 'e'->word[3]='l' != 'e' -> false ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
expand the abbreviation:
    each number n becomes n wildcard characters
    compare the expanded pattern with the word
        character by character
```

TIME	SPACE
O(n + m)	O(m)

NOTE

Not slow. Building the expanded pattern costs O(m) memory; two pointers that parse each number and skip that many characters of the word need O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Use pointer i into word and j into abbr.

When abbr[j] is a digit: check for leading zero (abbr[j]=='0' and num==0 -> invalid).

Accumulate the number: num = num*10 + digit.

When abbr[j] is a letter: first advance i by num (the accumulated skip), reset num to 0.

Then compare word[i] with abbr[j] — if they differ, return False. Advance both i and j.

PHASE 3 OPTIMIZE

After the loop: we're valid only if i + num == len(word) AND j == len(abbr).

That handles a trailing number at the end of abbr.

Pseudocode:

i, j, num = 0, 0, 0

```
while i < len(word) and j < len(abbr):
if abbr[j].isdigit():
if abbr[j]=='0' and num==0: return False # leading zero
num = num*10 + int(abbr[j])
else:
i += num; num = 0
if i >= len(word) or word[i] != abbr[j]: return False
i += 1
j += 1
return i + num == len(word) and j == len(abbr)
```

Time: O(n + m) where n=len(word), m=len(abbr). Space: O(1)."

Key insight: two pointers (i into word, j into abbr) advance with different rules — a digit batch in abbr advances i by the decoded number, catching leading-zero violations in the same step.

PHASE 4 CLEAN CODE

Now I'll implement with meaningful names.

```
class _408_ValidWordAbbreviation:
    def validWordAbbreviation(self, word: str, abbr: str)
-> bool:
        word_ptr = 0
        abbr_ptr = 0
        skip     = 0

        while word_ptr < len(word) and abbr_ptr < len(abbr
):
            if abbr[abbr_ptr].isdigit():
                if abbr[abbr_ptr] == '0' and skip == 0:
# leading zero
                    return False
                skip = skip * 10 + int(abbr[abbr_ptr])
            else:
                word_ptr += skip
                skip = 0
                if word_ptr >= len(word) or word[word_ptr]
 != abbr[abbr_ptr]:
                    return False
                word_ptr += 1
            abbr_ptr += 1

        return word_ptr + skip == len(word) and abbr_ptr =
= len(abbr)
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

"Let me trace both examples."

word='internationalization'(len=20), abbr='i12iz4n'

j=0: 'i' -> skip=0, advance word_ptr 0, word[0]='i'=='i' ok. word_ptr=1.

PHASE 5 TEST & DEBUG

j=1: '1' -> skip=1.

j=2: '2' -> skip=12.

j=3: 'i' -> word_ptr+=12=13. word[13]='i'=='i' ok. word_ptr=14.

j=4: 'z' -> skip=0, word[14]='z'=='z' ok. word_ptr=15.

j=5: '4' -> skip=4.

j=6: 'n' -> word_ptr+=4=19. word[19]='n'=='n' ok. word_ptr=20.

Loop ends (word_ptr=20=len(word)).
word_ptr+skip=20+0=20==20, abbr_ptr=7==7. True ok

word='apple'(len=5), abbr='a2e'

j=0: 'a' -> word[0]='a'=='a' ok. word_ptr=1.

j=1: '2' -> skip=2.

j=2: 'e' -> word_ptr+=2=3. word[3]='l' != 'e' -> return False ok

Leading zero: abbr='a01e' -> j=1: '0' and skip==0 -> return False ok

Leading zero check and trailing number handled correctly

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + m)	**O(1)**
each character in word and abbr is visited at most once	three integer pointers

The two trickiest cases to get right:

1. Leading zeros — must check abbr[j]=='0' AND accumulated num==0 to distinguish

'0' (leading zero, invalid) from '10', '20' etc. (valid multi-digit numbers ending in 0).

2. Trailing number — if abbr ends with digits (e.g. abbr='a4'), the loop exits

when abbr_ptr reaches end but skip is still non-zero.

The final check word_ptr + skip == len(word) handles this correctly.

Given more time I'd ask: what if we want to generate ALL valid abbreviations

```
for a word? That's a backtracking problem — enumerate
  all subsets of positions
```

to replace with their lengths.

IN YOUR APP | Expose | Community

EASY | Two Pointers | LeetCode #455 | 12 of 16 | 12/16

Assign Cookies

QUESTION

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1

Input	`g = [1,2,3], s = [1,1]`
Output	`1`
Explanation	*You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.*

Example 2

Input	`g = [1,2], s = [1,2,3]`
Output	`2`
Explanation	*You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2. You have 3 cookies and their sizes are big enough to gratify all of the children, You need to output 2.*

CONSTRAINTS

- $1 <= g.length <= 3 * 10^4$
- $0 <= s.length <= 3 * 10^4$
- $1 <= g[i], s[j] <= 2^{31} - 1$
- Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Assign cookies to children greedily: cookie size >= child's greed satisfies that child.

Maximise number of satisfied children. Right?"

"A few quick questions:

Each child gets at most one cookie, each cookie used at most once? Yes."

"Let me walk: g=[1,2,3], s=[1,1] -> assign s[0]=1 to g[0]=1 -> 1 satisfied ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for assignment in cookies_to_children:
    count the satisfied children
    best = max(best, candidate)
```

TIME	SPACE
O(n!)	O(n + m)

WHY IT IS SLOW

Tries every assignment. Sort both arrays and give each child the smallest cookie that satisfies them with two pointers: O(n log n + m log m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer greedy — O(n log n + m log m). Walk-through:"

Sort both g (greed factors) and s (cookie sizes).

i=0 (child index), j=0 (cookie index).

While i < n and j < m:

If s[j] >= g[i]: cookie j satisfies child i -> i++; j++.

Else: j++ (cookie too small for this child — skip it).

WHY not try smaller cookies for this child: cookies are sorted, so if s[j] < g[i],

all cookies s[0..j] are also < g[i]. Moving to s[j+1] is the only option.

WHY not try a larger child for this cookie: sorted children means g[i] is the smallest

unsatisfied greed. If s[j] can't satisfy g[i], it can't satisfy any remaining child.

Answer = i. Time: O(n log n + m log m). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _455_AssignCookies:
    def findContentChildren(self, g, s):
        g.sort()
        s.sort()
        child_ptr = cookie_ptr = 0
        while child_ptr < len(g) and cookie_ptr < len(s):
            if s[cookie_ptr] >= g[child_ptr]:
                child_ptr += 1
            cookie_ptr += 1
        return child_ptr
```

PHASE 5 TEST & DEBUG

"Let me trace assignCookies(g=[1,2,3], s=[1,1])."

Sort both: g=[1,2,3], s=[1,1]

gi=0,si=0: s[0]=1 >= g[0]=1 -> assign. gi=1,si=1

gi=1,si=1: s[1]=1 < g[2]=2 -> cookie too small. si=2 (exhausted). Stop.

Return gi=1 ok (1 child satisfied)

WHY greedy: try to satisfy least greedy child with smallest sufficient cookie.

Two-pointer: advance cookie pointer regardless; advance child only when satisfied.

Edge: no cookies -> return 0; all cookies > all greed -> return len(g).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) sort dominates	**O(1)**

Follow-up: if children have priorities, use a max-heap.

Follow-up: Candy (#135) — each child gets at least 1 candy, greedy with two passes.

EASY | Two Pointers | LeetCode #680 | 13 of 16 | 13/16

Valid Palindrome II

QUESTION

Given a string s, return true if the s can be palindrome after deleting at most one character from it.

Example 1	
Input	`s = "aba"`
Output	`true`

Example 2	
Input	`s = "abca"`
Output	`true`
Explanation	*You could delete the character 'c'.*

Example 3	
Input	`s = "abc"`
Output	`false`

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if we can make s a palindrome by deleting AT MOST one character. Right?"

"A few quick questions:

Zero deletions also valid? Yes — if already palindrome, return true.

Case-sensitive? Yes."

"Let me walk: s='aba' -> already palindrome ok. s='abca' -> delete 'c' -> 'aba' ok.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
if the string is a palindrome:
    return True
for i in range(n):
    delete that character and test whether the
        rest is a palindrome
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Rebuilds and checks the string for each deletion. Two pointers inward; at the first mismatch test only the two options (skip left or skip right): O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n²) try-every-deletion approach.

Two pointers from both ends. When mismatch found, we have one deletion budget:

`try skipping s[left] (check s[left+1..right] is palindrome) OR`

`try skipping s[right] (check s[left..right-1] is palindrome).`

If either is a palindrome, return True.

Time: O(n). Space: O(1)."

Key insight: with exactly one deletion budget, a mismatch at [left, right] yields exactly two candidate ranges — skip left or skip right; a single is_palindrome check per candidate resolves it in O(n) total.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _680_ValidPalindromeII:
    def validPalindrome(self, s):
        def is_pal(lo, hi):
            while lo < hi:
                if s[lo] != s[hi]: return False
                lo += 1; hi -= 1
            return True

        left, right = 0, len(s) - 1
        while left < right:
            if s[left] != s[right]:
                return is_pal(left + 1, right) or is_pal(l
eft, right - 1)
            left += 1; right -= 1
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

s='abca': left=0('a'),right=3('a') match. left=1('b'),right=2('c') mismatch.

`is_pal(2,2)=True (skip 'b') -> True ok`

s='abc': left=0('a'),right=2('c') mismatch.

`is_pal(1,2)='bc'->False. is_pal(0,1)='ab'->False. -> False ok`

We only get one mismatch chance — the first mismatch triggers the two attempts

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
at most two passes	only index variables

Follow-up: Valid Palindrome III (#1216) — at most k deletions; DP $O(n^2)$.

Follow-up: Minimum Deletions to Make String Palindrome — n - LPS (Longest Palindromic Subsequence)(s).

IN YOUR APP | Expose | Editorial | Community

EASY | Two Pointers | LeetCode #696 | 14 of 16 | 14/16

Count Binary Substrings

QUESTION

Given a binary string s, return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1

Input	`s = "00110011"`
Output	**6**
Explanation	*There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", and "01". Notice that some of these substrings repeat and are counted the number of times they occur. Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.*

Example 2

Input	`s = "10101"`
Output	**4**
Explanation	*There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s[i] is either '0' or '1'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count non-empty substrings that have equal numbers of 0s and 1s,

and all 0s and all 1s come in consecutive groups of the same length. Right?"

"A few quick questions:

'00110011' -> substrings: '0011','01','1100','10','001100','0110' etc. Count these? Yes."

"Let me walk: s='00110011' -> 6 ok. s='10101' -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    check that it has equal consecutive 0s and 1s
    count the valid ones
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) substrings, each checked in O(n). Group the string into runs of equal characters: each pair of neighbouring runs adds min(run1, run2) substrings, O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the O(n²) substring checking.

Key insight: adjacent groups of same characters. For any two consecutive groups,

the number of valid substrings bridging them = min(len(group1), len(group2)).

Compress s into run-length encoding, then sum min of consecutive pairs.

Time: O(n). Space: O(n) or O(1) with rolling approach.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _696_CountBinarySubstrings:
    def countBinarySubstrings(self, s):
        groups = []
        count = 1
        for i in range(1, len(s)):
            if s[i] == s[i-1]:
                count += 1
            else:
                groups.append(count)
                count = 1
        groups.append(count)
        return sum(min(groups[i], groups[i+1]) for i in ra
nge(len(groups)-1))
```

PHASE 5 TEST & DEBUG

"Let me trace countBinarySubstrings on '00110011'."

Group consecutive: [2,2,2,2] (two 0s, two 1s, two 0s, two 1s)

Count pairs from adjacent groups: min(2,2)+min(2,2)+min(2,2)=2+2+2=6 ok

('0011','01','10','1100','0011' wait—all valid: 00110011 has 6 substrings where 0s and 1s equal count consecutively)

PHASE 5 TEST & DEBUG

WHY group lengths: each adjacent pair contributes min(len_i, len_{i+1}) valid substrings.

Two-pass: first compute group lengths, then sum adjacent mins.

Alt O(n): two-pointer counting consecutive groups without building group array explicitly.

Groups of same-char runs: two-pointer scans s once; group boundaries clear without auxiliary array.

Groups=[2,2,2,2] for '00110011'. min(2,2)=2 pairs each transition. total=2+2+2=6 ok. Each group[i] and group[i+1] pair contributes min(group[i],group[i+1]) substrings.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** if we process groups on the fly without storing the array

The min formula comes from: a group of m 0s and n 1s contributes min(m,n) matching substrings.

Follow-up: if we want the actual substrings, track start indices during group iteration.

Follow-up: generalise to k distinct characters in groups.

EASY | Two Pointers | LeetCode #977 | 15 of 16 | 15/16

Squares of a Sorted Array

QUESTION

Given an integer array nums sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1	
Input	`nums = [-4,-1,0,3,10]`
Output	`[0,1,9,16,100]`
Explanation	*After squaring, the array becomes [16,1,0,9,100].* *After sorting, it becomes [0,1,9,16,100].*

Example 2	
Input	`nums = [-7,-3,2,3,11]`
Output	`[4,9,9,49,121]`

CONSTRAINTS

- $1 <= nums.length <= 10^4$
- $10^4 <= nums[i] <= 10^4$
- nums is sorted in non-decreasing order.
- Follow up: Squaring each element and sorting the new array is very trivial, could you find an O(n) solution using a different approach?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

The input array is sorted in non-decreasing order and can contain negatives.

We need to return a new array of the squares of each element, also sorted. Right?"

"A few quick questions:

Can the array contain negative numbers? Yes — that's the whole challenge.

Should I return a new array or modify in-place? Return a new array.

Can the array have a single element? Yes — just return [element²].

Can all elements be negative? Yes — e.g. [-3,-2,-1] -> [1,4,9]."

"Let me walk example 1:

nums=[-4,-1,0,3,10]. Squares: [16,1,0,9,100]. Sorted: [0,1,9,16,100].

The key observation: the largest squares come from the extremes —

either the most negative or the most positive end. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
square every number
sort the squares
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

The sort is unnecessary because the input is sorted. Two pointers from both ends place the larger square at the end of the output each step: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force ignores the fact that the input is already sorted.

Key insight: in a sorted array, the largest absolute values are always at one

of the two ends — either the most negative (left) or the most positive (right).

So the largest square is always max(nums[left]², nums[right]²).

Use two pointers: left starts at 0, right at n-1.

Fill the result array from the back (largest to smallest):

at each step, compare abs(nums[left]) and abs(nums[right]).

Place the larger square at result[k] and move that pointer inward.

Pseudocode:

```
result = [0] * n; k = n-1; left, right = 0, n-1
while k >= 0:
if abs(nums[left]) > abs(nums[right]):
result[k] = nums[left]**2; left += 1
else:
result[k] = nums[right]**2; right -= 1
k -= 1
return result
```

Time: O(n). Space: O(n) for the output — unavoidable since we return a new array.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _977_SquaresSortedArray:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n      = len(nums)
        result = [0] * n
        fill   = n - 1
        left   = 0
        right  = n - 1

        while fill >= 0:
```

```
            if abs(nums[left]) > abs(nums[right]):
                result[fill] = nums[left] * nums[left]
                left += 1
            else:
                result[fill] = nums[right] * nums[right]
                right -= 1
            fill -= 1

        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

"Let me trace both examples."

nums=[-4,-1,0,3,10], n=5

fill=4: abs(-4)=4 < abs(10)=10 -> result[4]=100, right=3. fill=3.

fill=3: abs(-4)=4 > abs(3)=3 -> result[3]=16, left=1. fill=2.

fill=2: abs(-1)=1 < abs(3)=3 -> result[2]=9, right=2. fill=1.

fill=1: abs(-1)=1 > abs(0)=0 -> result[1]=1, left=2. fill=0.

fill=0: left==right==2, abs(0)=0 -> result[0]=0, right=1. fill=-1.

result=[0,1,9,16,100] ok

nums=[-7,-3,2,3,11], n=5

fill=4: abs(-7)=7 < abs(11)=11 -> result[4]=121, right=3. fill=3.

fill=3: abs(-7)=7 > abs(3)=3 -> result[3]=49, left=1. fill=2.

fill=2: abs(-3)=3 == abs(3)=3 -> take right: result[2]=9, right=2. fill=1.

fill=1: abs(-3)=3 > abs(2)=2 -> result[1]=9, left=2. fill=0.

fill=0: left==right==2 -> result[0]=4. fill=-1.

result=[4,9,9,49,121] ok

Tie goes to right which is fine — both are equal

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
single pass from both ends meeting in the middle	required for the output; we can't avoid this since we return a new array

This is the same fill-from-back pattern as Merge Sorted Array (#88) —

worth naming that explicitly to show you recognise the pattern family.

Trade-off: the brute force O(n log n) is two lines and perfectly acceptable

in most interviews. The O(n) two-pointer version is the follow-up answer

the problem explicitly asks for.

Given more time I'd ask: what if we needed to do this in-place?

Squaring in-place destroys order for negative numbers, so we'd need to

PHASE 6 COMPLEXITY & FOLLOW-UP

square + reverse the negative half, then merge the two sorted halves

in-place — doable but significantly more complex.

EASY | Two Pointers | LeetCode #1768 | 16 of 16 | 16/16

Merge Strings Alternately

QUESTION

You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return the merged string.

Example 1

Input	`word1 = "abc", word2 = "pqr"`
Output	`"apbqcr"`
Explanation	*The merged string will be merged as so:* *word1: a b c* *word2: p q r* *merged: a p b q c r*

Example 2

Input	`word1 = "ab", word2 = "pqrs"`
Output	`"apbqrs"`
Explanation	*Notice that as word2 is longer, "rs" is appended to the end.* *word1: a b* *word2: p q r s* *merged: a p b q r s*

Example 3

Input	`word1 = "abcd", word2 = "pq"`
Output	`"apbqcd"`
Explanation	*Notice that as word1 is longer, "cd" is appended to the end.* *word1: a b c d* *word2: p q* *merged: a p b q c d*

CONSTRAINTS

- 1 <= word1.length, word2.length <= 100
- word1 and word2 consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Merge two strings by alternating characters, starting with word1.

If one string runs out, append the remaining characters. Right?"

"A few quick questions:

They can be different lengths? Yes. Return a new string? Yes."

"Let me walk: word1='abc', word2='pqr' -> 'apbqcr' ok. word1='ab', word2='pqrs' -> 'apbqrs' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
i = 0
while either string has letters left:
    append word1[i] if it exists, then word2[i]
        if it exists
    i = i + 1
```

TIME	SPACE
O(n + m)	O(n + m)

NOTE

Not slow: each letter is visited once, which is optimal. The only extra is the output string.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer alternating picks — O(n+m). Walk-through:"

result=[]. i=0 (ptr in word1). j=0 (ptr in word2).

While i < len(word1) or j < len(word2):

If i < len(word1): result.append(word1[i]); i++.

If j < len(word2): result.append(word2[j]); j++.

Return ''.join(result).

WHY two separate ifs (not if-else): take from word1, then from word2 each iteration.

This gives the alternating pattern: w1[0], w2[0], w1[1], w2[1], ...

After one string is exhausted: remaining chars of the other are appended.

Time: O(n+m). Space: O(n+m) for result."

Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1768_MergeStringsAlternately:
    def mergeAlternately(self, word1, word2):
        result = []
        for c1, c2 in zip(word1, word2):
            result.append(c1)
            result.append(c2)
        result.append(word1[len(word2):])
        result.append(word2[len(word1):])
        return ''.join(result)
```

PHASE 5 TEST & DEBUG

"Let me trace how the left/right pointers move and why the invariant stays true."

word1='ab', word2='pqrs': zip pairs: a,p then b,q. remainder: word2[2:]='rs'.

PHASE 5 TEST & DEBUG

-> 'apbqrs' ok

word1='abcd', word2='pq': zip: a,p then b,q. remainder: word1[2:]='cd'.

-> 'apbqcd' ok

word1='a', word2='b': zip: a,b. no remainder. -> 'ab' ok

The slicing at the end appends the correct remainder from whichever string is longer

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(n+m)** for the result

Cleaner one-liner: ''.join(c for pair in zip_longest(word1,word2,'') for c in pair).

Follow-up: if strings can be very long, use a generator to avoid materializing the result.

Follow-up: merge k strings alternately — use itertools.roundrobin pattern.